Database Administration
Mr. John Kenneth Timblaco
BSIT 3-2

DropBoxed

Activity 2 - DATABASE SETUP & APPLICATION DATABASE BLUEPRINT

Prepared by **RIPARION**

Abalo, Joshua
Dela Cruz, Jan Emmanuel A.
Reyes, Denver R.
Santiago, Jhayciel D.
Valera, Rhaine Cyrel M.



Activity 2 - DATABASE SETUP & APPLICATION DATABASE BLUEPRINT

This activity serves as the planning and setup stage of your group's database project. You will establish and verify a PostgreSQL environment, propose an application, analyze its requirements, define business rules, develop its database design, and document the planned system architecture. The goal is not to rush into coding. The goal is to produce a design that is clear enough to guide implementation and future database administration.

Your group database will become a working database environment that can be used in subsequent Database Administration activities.

Learning Objectives

- Set up and verify a PostgreSQL database environment.
- Apply the database design process to a proposed application.
- Analyze requirements and translate them into data requirements and business rules.
- Develop a conceptual database model using an ERD.
- Translate the conceptual model into a relational schema.
- Apply normalization and integrity constraints to the proposed design.
- Plan the PostgreSQL database structure and initial SQL/DDL implementation.
- Design and justify an appropriate application architecture.
- Document the database so that another person can understand and manage it.

Project Principle

PLAN FIRST. BUILD SECOND. Your database design and architecture must be documented and approved before substantial application development.

Important: AI-Assisted Development

AI tools may be used in this project. However, AI may assist with implementation; it does not replace your responsibility for the system design.

- You may use AI to generate source code, SQL, boilerplate, debugging suggestions, documentation drafts, or test data.
- The group must decide and document the requirements, business rules, ERD, relational schema, normalization, constraints, architecture, and technology choices.
- All AI-generated code must be reviewed, tested, and understood by the group.
- The group must disclose how AI was used in the project.The group must disclose how AI was used in the project.
- During consultation or checking, members may be asked to explain their database design or implementation.

PART 1 — POSTGRESQL SETUP & VERIFICATION

Complete the following checklist for the group's development environment. Students may use Windows, Linux, macOS, or an available laboratory computer. If PostgreSQL is already installed, verify the existing installation instead of reinstalling it unnecessarily.

A. Before Installation / Environment Planning

Task / Requirement	Status	Evidence / Notes
Operating system identified	Completed	Windows 11
PostgreSQL version selected/documented	Completed	PostgreSQL 16
CPU and RAM considered	Completed	Intel Core i5, 8GB RAM - Sufficient
Storage location considered	Completed	C:\Program Files\PostgreSQL\18\bin
PostgreSQL port identified	Completed	Port 5432
Installation/data location documented	Completed	Local Drive
Intended environment identified	Completed	Personal Computer (Localhost)

B. Installation and Verification

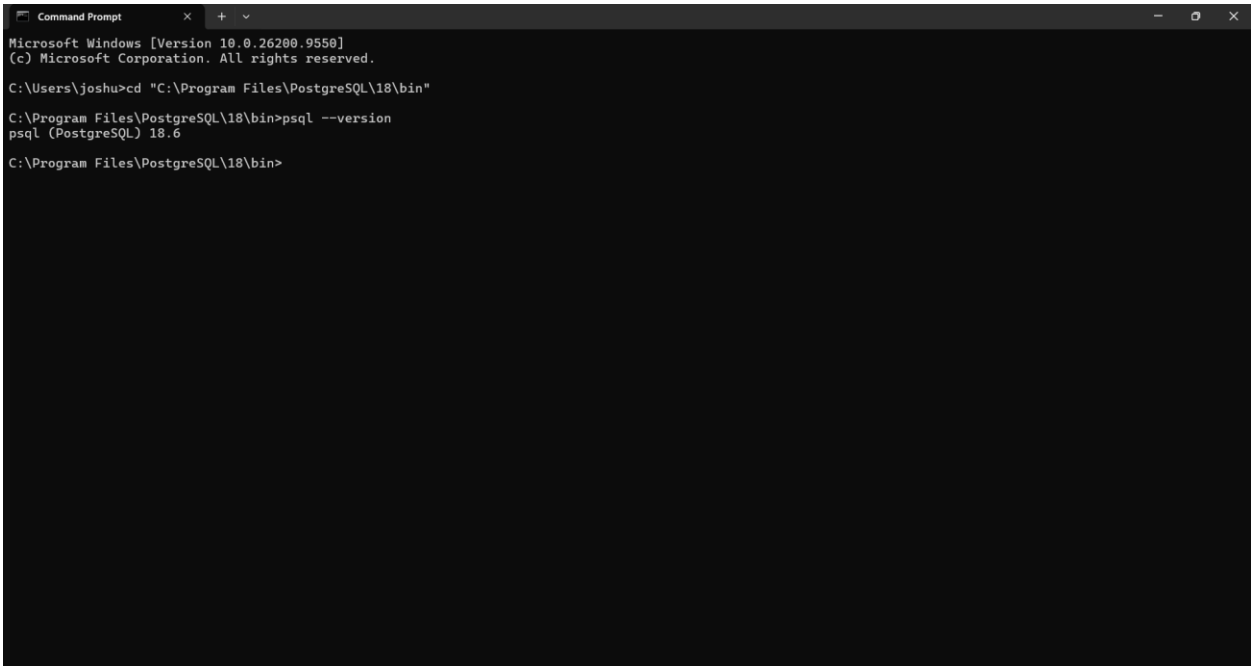
Task / Requirement	Status	Evidence / Notes
PostgreSQL installed or existing installation verified	Completed	Verified via pgAdmin 4

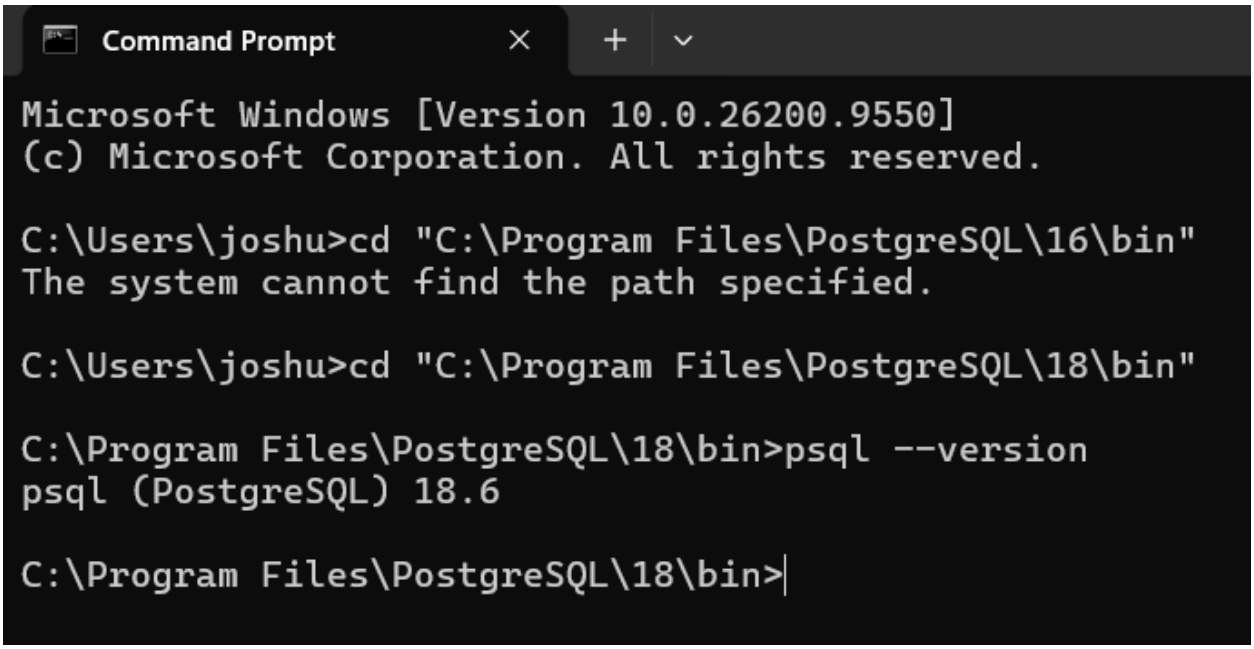
PostgreSQL service is running	Completed	Checked in Windows Services (services.msc)
psql command-line client is accessible	Completed	Tested via CMD
PostgreSQL version checked using psql --version	Completed	Returned psql (PostgreSQL) 18.6
Connection to PostgreSQL server established	Completed	Successfully connected locally
PostgreSQL server version verified using SELECT version();	Completed	PostgreSQL 16.x compiled by Visual C++
Database creation capability verified	Completed	Test database created and dropped
Connection to the project database verified	Completed	Connected to newly created dropboxed_db

The Lesson 2 installation workflow specifically emphasizes considering the operating system, PostgreSQL version, resources, storage, port, installation/data location, and intended environment before installation; after installation, students verify psql, the service, a connection, and SELECT version().

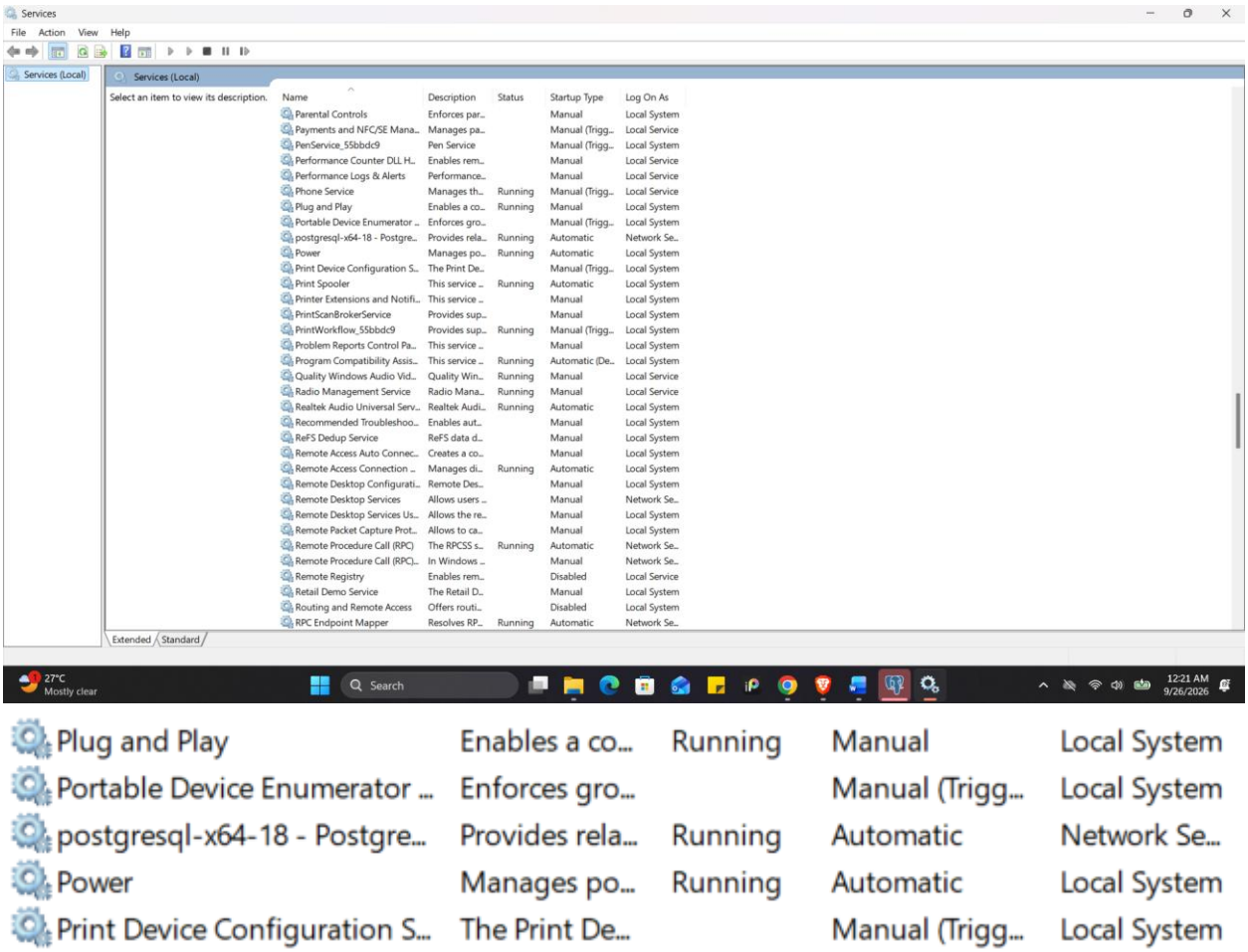
C. Evidence Required

- **Screenshot of PostgreSQL version verification.**

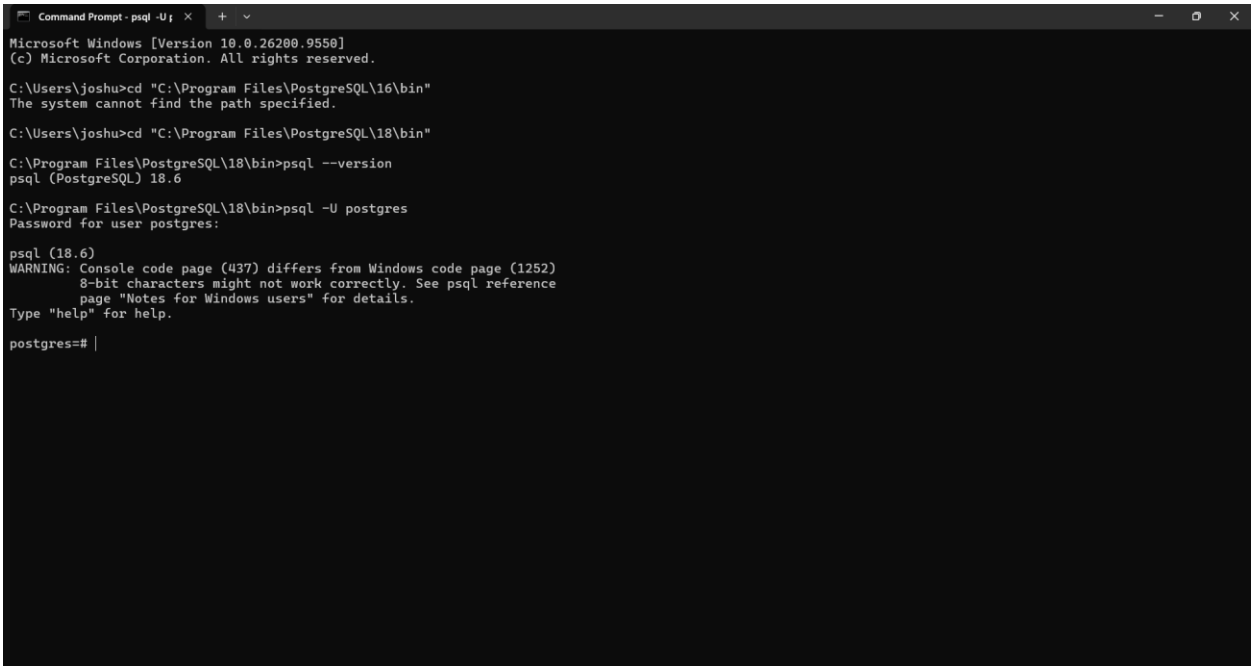


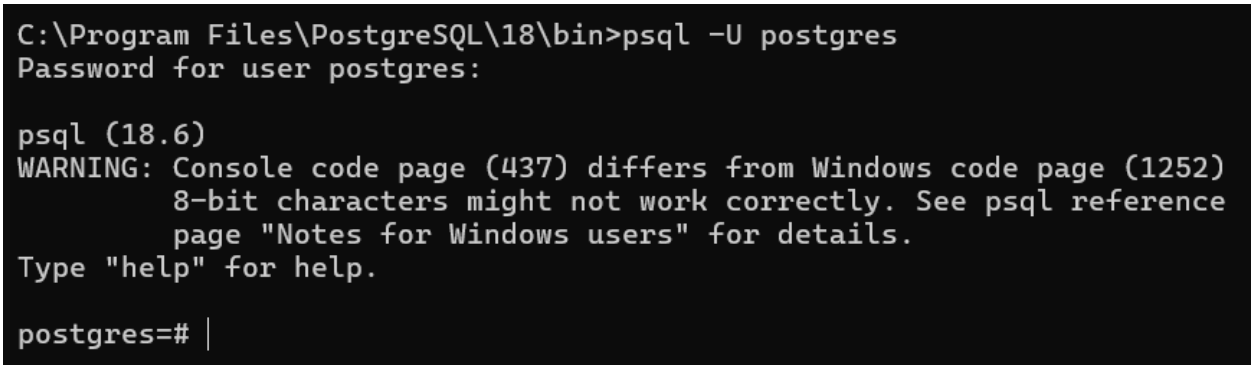


- **Screenshot showing the PostgreSQL service running.**

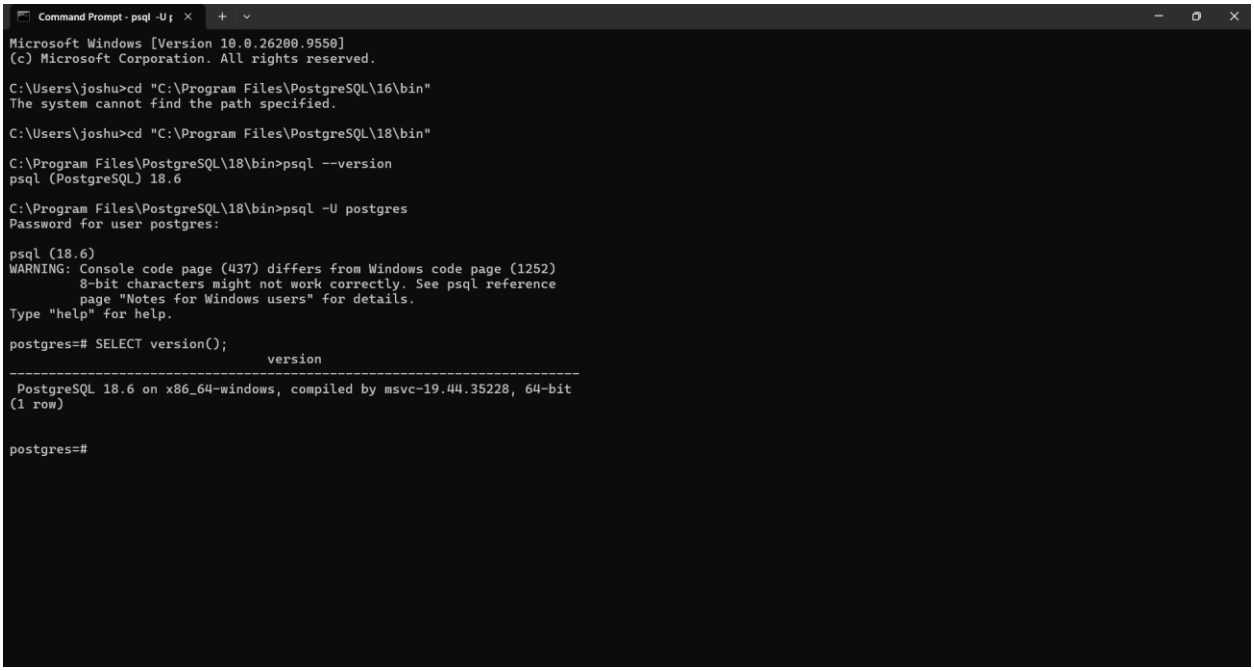


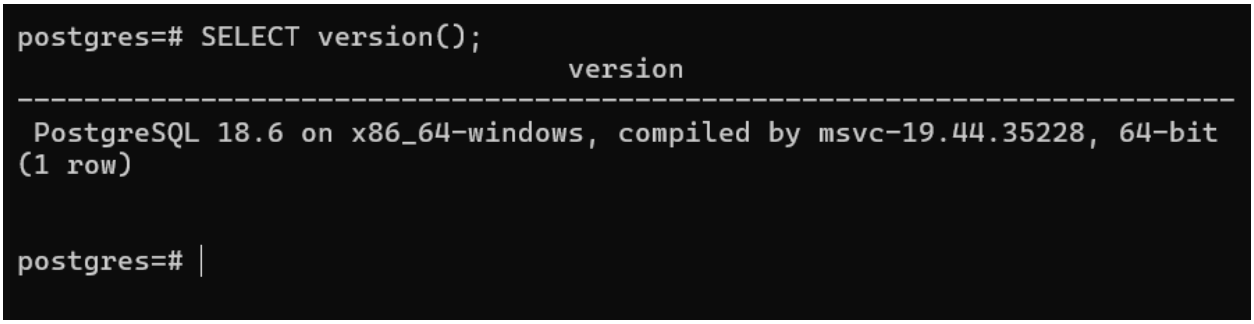
- Screenshot of a successful psql connection.



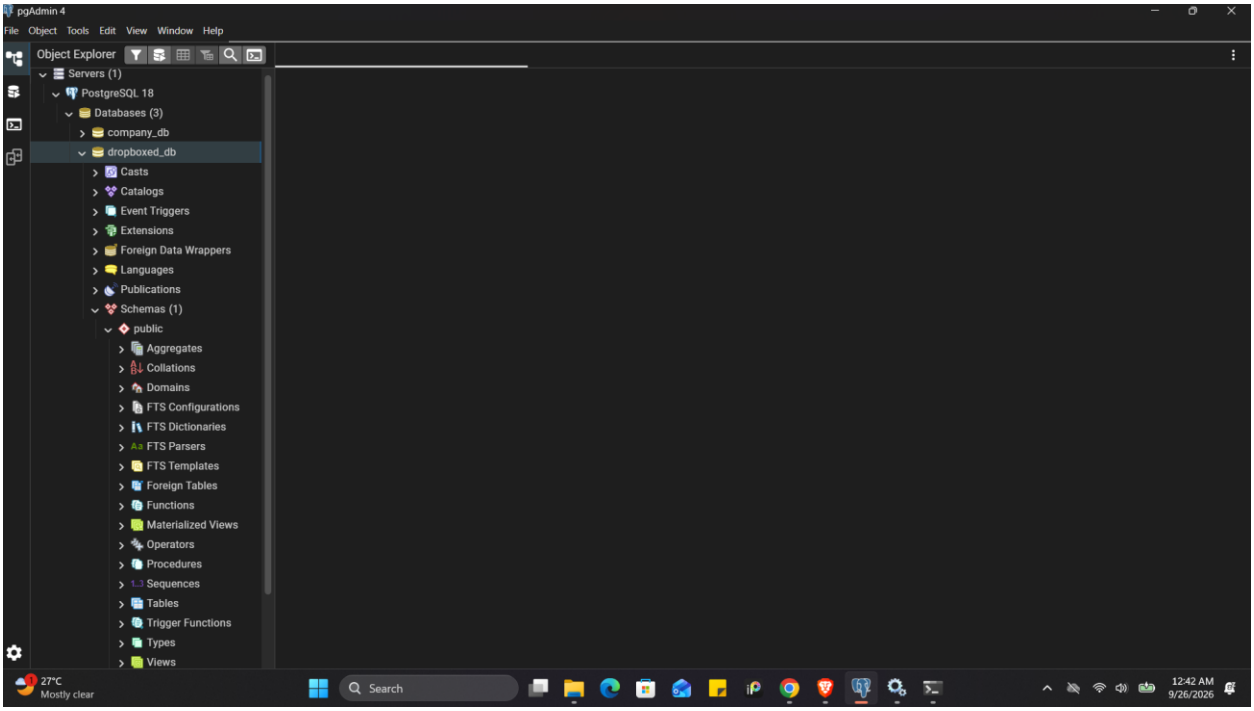


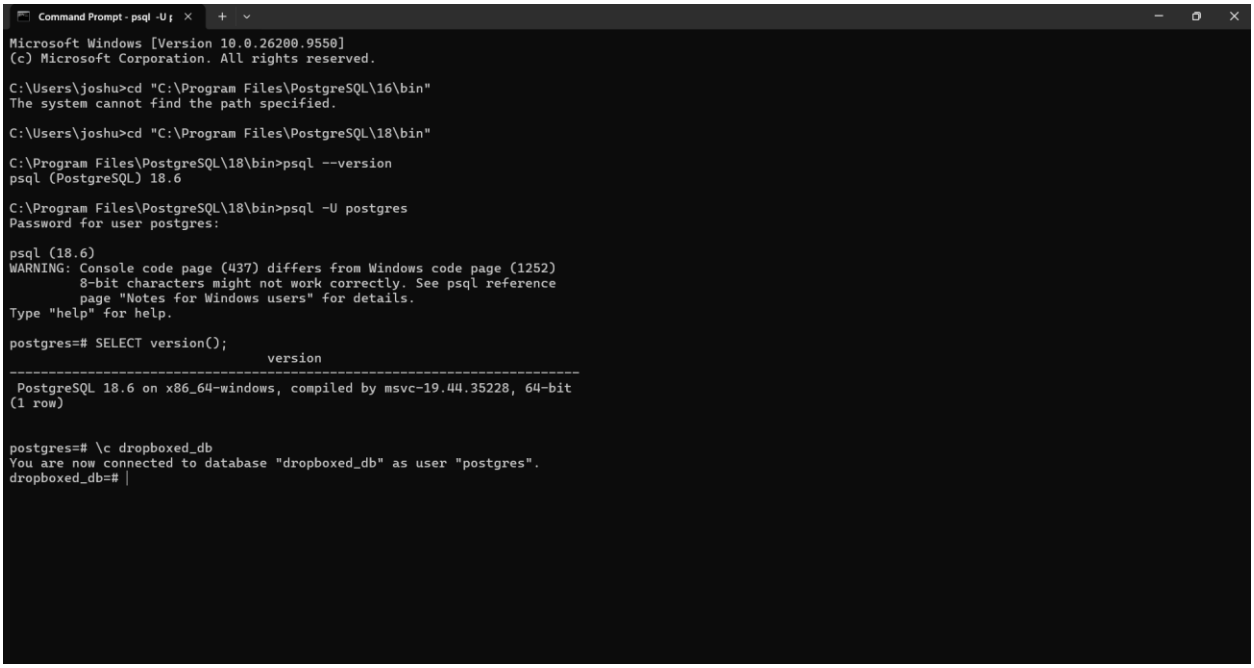
- **Screenshot showing SELECT version(); executing successfully.**

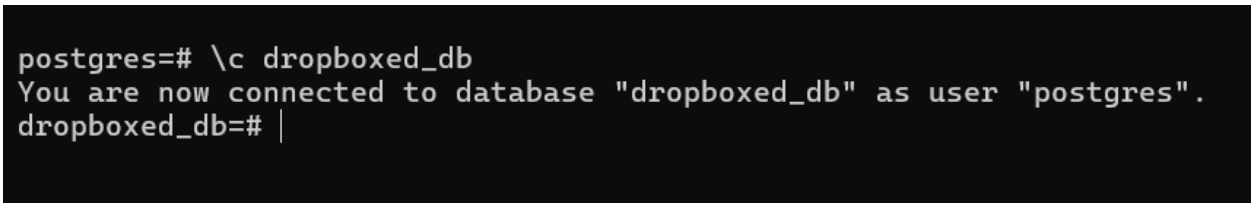




- **Screenshot showing the project database exists and can be accessed.**







Do not include passwords or other sensitive credentials in screenshots.

PART 2 — APPLICATION PROPOSAL

Choose an application that requires a relational database. The group may propose its own idea. Original ideas are encouraged as long as the scope is realistic for the group and suitable for PostgreSQL.

Minimum Application Requirements

- The application solves a recognizable problem or supports a real process.
- The application has identifiable users.
- The application stores persistent data.
- The application contains multiple related entities.
- The application supports meaningful CRUD operations.
- The application has identifiable business rules.
- The database can be implemented using PostgreSQL.
- The scope is realistic for the available time and group size.

A. Project Proposal

Application Name: DropBoxed: An Environmental Complaint Application for Household Water Quality Monitoring in San Pedro

Problem Statement: Many households in San Pedro experience water-related issues such as rusty or contaminated water, low water pressure, foul odor, and pipe leaks — often caused by old, deteriorating pipelines. Rusty and contaminated water in particular may contain lead, bacteria, or sediment, posing health risks for drinking, cooking, and bathing. Currently, residents have no centralized, trackable way to report these issues to their local water district, which leads to delayed responses and unresolved problems.

General Objective: To develop a digital platform that allows residents to report, track, and manage water-related complaints (with a focus on rusty and contaminated pipe water) so that local water district authorities can respond quickly and transparently.

Target Users:

Residents/Homeowners in San Pedro — submit complaints about water issues in their households

Water District Administrators/Staff (e.g., Primewater San Pedro) — review, update, and resolve complaints, and generate reports

Scope / Boundaries:

- Water complaints related to household pipes (rusty water, low pressure, foul odor, leaks), with primary focus on rusty/contaminated water from old pipelines
- User registration and login
- Complaint submission with category, description, and location
- Complaint status tracking (Pending → In Progress → Resolved)
- Admin interface for viewing all complaints, updating status, and generating reports

Out of scope:

- Water complaints unrelated to household pipes (e.g., billing issues, meter disputes)
- Real-time chat/live support between residents and district staff
- Payment processing or service fee handling
- Integration with external government or utility systems beyond Primewater

B. Proposed Features

List at least 8 major functional requirements/features.

#	Functional Requirement / Feature	Priority
1.User Registration & Login	residents can create an account and log in securely	High
2. Submit Complaint	users can file a complaint under a category (e.g. Water Pollution → Rusty Pipe Water, Low Pressure, Foul Odor, Pipe Leak)	High
3. Add Description & Location	users attach details and a location to each complaint for faster identification	High
4. View Complaint Status	users can check the current status of their submitted complaints (Pending, In Progress, Resolved)	High
5. Admin: View All Complaints	admins can see a list of all complaints with filtering/sorting options	High
6. Admin: Update Complaint Status	admins can change a complaint's status as it moves through Pending → In Progress → Resolved	High
7. Admin: Generate Reports	admins can generate summary reports (e.g., complaints by area, type, or status) for the water district	Medium
8. Complaint History / Tracking Log	the system keeps a timestamped record of status changes for each complaint	Medium

PART 3 — REQUIREMENTS ANALYSIS

Requirements analysis determines what the database needs to represent and what operations it needs to support. Do not rely entirely on assumptions. Consider how information would actually be collected, stored, updated, and used.

A. Requirements Sources

Task / Requirement	Status	Evidence / Notes
Interviews or consultation with intended users	Done	Informal discussion with household members in San Pedro about recurring water issues (rusty water, low pressure, leaks)
Existing documents	Not used	
Business forms	Not used	
Spreadsheets	Not used	
Reports	Done	Reviewed general information on Primewater San Pedro's service area and common complaint types
Existing applications	Done	Reviewed similar complaint/ticketing apps (e.g., barangay complaint systems, utility report apps) for reference on status tracking flow
Observation	Done	Observed common household water problems reported by residents (discoloration, odor, pressure drops)
Subject-matter expert discussion	Not used	

Identify the sources your group actually used:

Interviews/consultation with residents, review of existing complaint-tracking applications, and observation of common household water issues in San Pedro. These gave the group a realistic picture of what a resident needs to report and what an admin needs to track and resolve.

B. Data Requirements

Identify the major categories of information the system must store.

1. User Account Information
2. Complaint / Incident Details
3. Water Quality Issue Categories
4. Geographical / Location Data
5. Resolution & Status Logs
6. Water District Staff / Administrator Profiles
7. Media Attachments (photographic proof of pipe contamination)
8. Analytical Reports & System Records

Explain why each major data category is necessary to support the application's requirements.

1. User Account Information: Captures resident identity and login credentials to manage individual access and track personal submissions.

2. Complaint / Report Details: Forms the core records, capturing incident dates and specific descriptions of reported water quality issues.

3. Complaint Category: Categorizes issues such as rusty water, odor, or pressure drops to enable filtering and prioritized administrative handling.

4. Location Information: Connects reports to specific household addresses to guide field team dispatch and intervention.

5. Complaint Status History: Logs the resolution lifecycle from Pending to Resolved, establishing accountability and operational transparency.

6. Admin/Staff Information: Records assigned personnel and status updates to streamline internal workload management.

7. Attachments: Stores photo evidence (such as discolored water samples) to validate incidents and determine urgency prior to site visits.

8. Reports/Summary Records: Compiles aggregated analytics to reveal localized trends, helping identify recurring pipeline infrastructure failures.

PART 4 — BUSINESS RULES

Write at least 8 business rules. Each rule should describe how data should behave in the real-world process. Your rules should help determine relationships, cardinality, keys, foreign keys, and constraints.

#	Business Rule	Design Impact
1	Each resident must have one unique user account and may submit multiple water-related complaints.	Establishes a **1:N relationship** between users and complaints; `complaints.user_id` is an FK.
2	Each complaint must be submitted by exactly one registered resident and must contain a valid water issue category, location, and description.	`complaints.user_id`, `category_id`, `location_id`, and description should be **NOT NULL**; foreign keys ensure referenced records exist.
3	Each complaint must belong to one water issue category, such as rusty/contaminated water, low pressure, foul odor, or pipe leak.	Establishes a **1:N relationship** between `complaint_categories` and `complaints`; `complaints.category_id` is an FK.
4	Each complaint must identify the household location where the water issue was reported.	Establishes a relationship between `complaints` and `locations`; `complaints.location_id` is an FK.
5	A complaint must follow the defined status lifecycle of **Pending → In Progress → Resolved**.	`complaints.status` requires a controlled set of values through a CHECK constraint, while `status_logs` records status changes.
6	A complaint may be assigned to one or more authorized staff members, and a staff member may be assigned to multiple complaints.	Establishes an **M:N relationship** between `complaints` and `staff`, resolved using the associative table `complaint_assignments`.
7	Every status update must identify the complaint, the staff member responsible for the update, the new status, and the date and time of the update.	Requires `status_logs` with foreign keys to complaints and staff, plus a timestamp.
8	A complaint may contain multiple photographic attachments, but each attachment must belong to exactly one complaint.	Establishes a **1:N relationship** between `complaints` and `attachments`; `attachments.complaint_id` is an FK.
9	Uploaded attachments must use an accepted image format and must not exceed the application's allowed file size. Attachments that are invalid or unrelated to the complaint may be rejected or flagged during administrative review before being stored.	`attachments` stores file metadata such as `file_type` and `file_size_bytes`; application validation handles file format/size, while administrators handle relevance.
10	A resident may submit multiple legitimate complaints, but repeated submissions representing the same issue may be identified as potential duplicate or spam complaints and flagged for administrative review.	`complaints` needs fields such as `is_flagged` and `flag_reason`; duplicate detection is handled by application logic using information such as resident, category, location, and submission time.

11	A flagged complaint remains part of the system's records and must not be automatically deleted solely because it is suspected to be invalid, duplicated, or spam.	Preserves complaint history and supports administrative review and accountability.
12	User and staff accounts must have unique email addresses and an account status indicating whether the account is active.	`users.email` and `staff.email` require UNIQUE; an account status field can use a controlled value such as `Active` or `Inactive`.

PART 5 — CONCEPTUAL DATABASE DESIGN

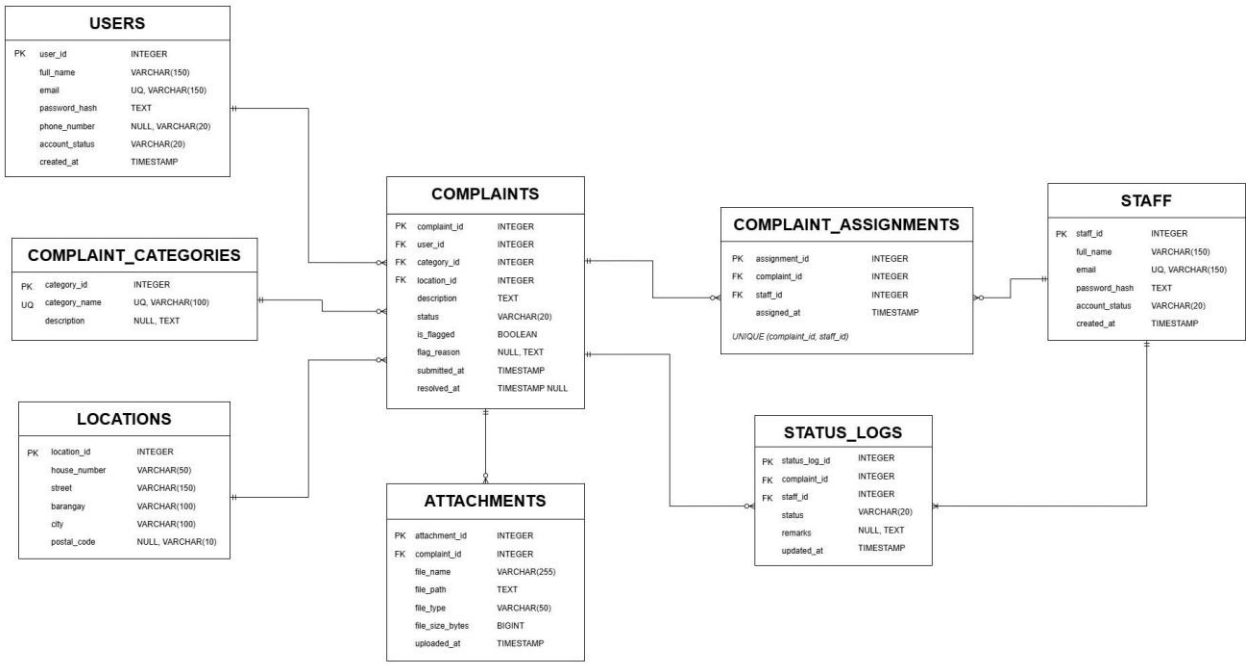
Create an ERD for your proposed application. The ERD should represent the entities, attributes, relationships, and cardinalities identified from your requirements and business rules.

Minimum ERD Requirements

- At least 5 meaningful entities.
- At least 1 one-to-many (1:N) relationship.
- At least 1 many-to-many (M:N) relationship.
- Primary keys identified.
- Foreign-key relationships identified where applicable.
- Cardinality clearly shown.
- Associative entity/table used where needed to resolve an M:N relationship.
- The ERD must be consistent with the requirements and business rules.

ERD

Insert your ERD below. Use a separate page if necessary.



ERD Design Notes

Explain three important design decisions made in your ERD.

1. Decision: Separate COMPLAINT_CATEGORIES and LOCATIONS from COMPLAINTS.

Reason: This avoids repeating category and location details in every complaint and reduces redundancy and update anomalies.

2. Decision: Use COMPLAINT_ASSIGNMENTS as an associative entity between COMPLAINTS and STAFF

Reason: A complaint can be assigned to multiple staff members, while a staff member can handle multiple complaints, so the M:N relationship needs an associative table.

3. Decision: Use separate STATUS_LOGS and ATTACHMENTS tables.

Reason: A complaint can have multiple status updates and multiple photos, so storing them in separate tables avoids repeating groups and allows the number of records to grow without changing the COMPLAINTS table.

PART 6 — RELATIONAL SCHEMA

Translate the conceptual design into relational schemas. Clearly identify primary keys, foreign keys, important nullable fields, and important uniqueness requirements.

Use the following format as a guide:

```
CUSTOMER
customer_id PK
name
email UQ

ORDER
order_id PK
customer_id FK
order_date
status
```

Your relational schemas:

USERS

user_id PK

full_name NOT NULL

email UQ NOT NULL

password_hash NOT NULL

phone_number NULL

account_status NOT NULL

created_at NOT NULL

STAFF
staff_id PK
full_name NOT NULL
email UQ NOT NULL
password_hash NOT NULL
account_status NOT NULL
created_at NOT NULL

COMPLAINT_CATEGORIES
category_id PK
category_name UQ NOT NULL
description NULL

LOCATIONS
location_id PK
house_number NOT NULL
street NOT NULL
barangay NOT NULL
city NOT NULL
postal_code NULL

COMPLAINTS
complaint_id PK
user_id FK NOT NULL
category_id FK NOT NULL
location_id FK NOT NULL
description NOT NULL
status NOT NULL
is_flagged NOT NULL
flag_reason NULL
submitted_at NOT NULL
resolved_at NULL

COMPLAINT_ASSIGNMENTS
assignment_id PK
complaint_id FK NOT NULL
staff_id FK NOT NULL
assigned_at NOT NULL
UNIQUE (complaint_id, staff_id)

STATUS_LOGS
status_log_id PK
complaint_id FK NOT NULL
staff_id FK NOT NULL
status NOT NULL
remarks NULL
updated_at NOT NULL

ATTACHMENTS
attachment_id PK
complaint_id FK NOT NULL
file_name NOT NULL
file_path NOT NULL
file_type NOT NULL
file_size_bytes NOT NULL
uploaded_at NOT NULL

Schema Consistency Check

- Every table has an appropriate primary key.
- Foreign keys correspond to referenced primary keys.
- M:N relationships have been resolved appropriately.
- Attributes are assigned to the table/entity they describe.
- The relational schema is consistent with the ERD.
- The schema supports the stated requirements and business rules.

PART 7 — DATA DICTIONARY

Complete a data dictionary for every table in your proposed database.

TABLE: USERS

Column	Data Type	PK	FK	Nullable	Descriptions
user_id	INTEGER	✓	—	No	Unique identifier of the resident account.
full_name	VARCHAR(150)	—	—	No	Full name of the resident.

email	VARCHAR(150)	—	—	No	Unique email address used for account identification/login.
password_hash	TEXT	—	—	No	Securely hashed password for authentication.
phone_number	VARCHAR(20)	—	—	Yes	Contact number of the resident.
account_status	VARCHAR(20)	—	—	No	Indicates whether the account is Active or Inactive.
created_at	TIMESTAMP	—	—	No	Date and time when the account was created.

TABLE: STAFF

Column	Data Type	PK	FK	Nullable	Descriptions
staff_id	INTEGER	✓	—	No	Unique identifier of the staff/admin account.
full_name	VARCHAR(150)	—	—	No	Full name of the staff member.
email	VARCHAR(150)	—	—	No	Unique email address of the staff member.
password_hash	TEXT	—	—	No	Securely hashed password for staff authentication.
account_status	VARCHAR(20)	—	—	No	Indicates whether the staff account is Active or Inactive.
created_at	TIMESTAMP	—	—	No	Date and time when the staff account was created.

TABLE: COMPLAINT_CATEGORIES

Column	Data Type	PK	FK	Nullable	Descriptions
category_id	INTEGER	✓	—	No	Unique identifier of the complaint category.

category_name	VARCHAR(100)	—	—	No	Name of the water issue category.
description	TEXT	—	—	Yes	Description of the type of water issue.

TABLE: LOCATIONS

Column	Data Type	PK	FK	Nullable	Descriptions
location_id	INTEGER	✓	—	No	Unique identifier of the reported household location.
house_number	VARCHAR(50)	—	—	No	House or building number.
street	VARCHAR(150)	—	—	No	Street where the affected household is located.
barangay	VARCHAR(100)	—	—	No	Barangay of the reported location.
city	VARCHAR(100)	—	—	No	City of the reported location.
postal_code	VARCHAR(10)	—	—	Yes	Postal code of the reported location.

TABLE: COMPLAINTS

Column	Data Type	PK	FK	Nullable	Descriptions
complaint_id	INTEGER	✓	—	No	Unique identifier of each complaint.
user_id	INTEGER	—	✓	No	Resident who submitted the complaint.
category_id	INTEGER	—	✓	No	Water issue category of the complaint.
location_id	INTEGER	—	✓	No	Household location associated with the complaint.

description	TEXT	—	—	No	Detailed description of the reported water issue.
status	VARCHAR(20)	—	—	No	Current status: Pending, In Progress, or Resolved.
is_flagged	BOOLEAN	—	—	No	Indicates whether the complaint has been flagged for administrative review.
flag_reason	TEXT	—	—	Yes	Reason why the complaint was flagged, such as suspected duplicate or invalid information.
submitted_at	TIMESTAMP	—	—	No	Date and time the complaint was submitted.
resolved_at	TIMESTAMP	—	—	Yes	Date and time the complaint marked as resolved.

TABLE: COMPLAINT_ASSIGNMENTS

Column	Data Type	PK	FK	Nullable	Descriptions
assignment_id	INTEGER	✓	—	No	Unique identifier of the assignment record.
complaint_id	INTEGER	—	✓	No	Complaint assigned to the staff member.
staff_id	INTEGER	—	✓	No	Staff member assigned to the complaint.
assigned_at	TIMESTAMP	—	—	No	Date and time when the complaint was assigned.

TABLE: STATUS_LOGS

Column	Data Type	PK	FK	Nullable	Descriptions
status_log_id	INTEGER	✓	—	No	Unique identifier of the status history record.
complaint_id	INTEGER	—	✓	No	Complaint associated with the status update.
staff_id	INTEGER	—	✓	No	Staff member who performed the status update.
status	VARCHAR(20)	—	—	No	Status assigned during the update.
remarks	TEXT	—	—	Yes	Optional remarks regarding the status update.
updated_at	TIMESTAMP	—	—	No	Date and time of the status update.

TABLE: ATTACHMENTS

Column	Data Type	PK	FK	Nullable	Descriptions
attachment_id	INTEGER	✓	—	No	Unique identifier of the uploaded attachment.
complaint_id	INTEGER	—	✓	No	Complaint associated with the attachment.
file_name	VARCHAR(255)	—	—	No	Name of the uploaded image file.
file_path	TEXT	—	—	No	Storage path used by the application to locate the uploaded image.
file_type	VARCHAR(50)	—	—	No	MIME/file type of the attachment, such as image/jpeg or image/png.
file_size_bytes	BIGINT	—	—	No	Size of the uploaded file in bytes.

uploaded_at	TIMESTAMP	—	—	No	Date and time the attachment was uploaded.

PART 8 — NORMALIZATION REVIEW

Review the proposed design for unnecessary redundancy and modification anomalies. Briefly explain how the design addresses 1NF, 2NF, and 3NF. A formal decomposition algorithm is not required.

1NF — First Normal Form

Are values atomic? Are there repeating groups or multi-valued fields that should be separated?

- Yes, the design satisfies 1NF. Every attribute holds a single atomic value, with no comma separated or repeating fields. The design specifically avoids repeating groups by pulling anything that could recur into its own table: multiple photos per complaint go into ATTACHMENTS instead of photo1/photo2/photo3 columns, multiple staff assignments per complaint go into COMPLAINT_ASSIGNMENTS instead of a list of staff IDs, and the history of status changes goes into STATUS_LOGS instead of status1/date1, status2/date2 columns in COMPLAINTS.

2NF — Second Normal Form

For tables with composite keys, do non-key attributes depend on the whole key?

- None of the tables use a composite primary key. Even COMPLAINT_ASSIGNMENTS, which has a natural composite key of (complaint_id, staff_id), uses a single-column surrogate key (assignment_id) instead and enforces the natural key as a UNIQUE constraint. Since every table's PK is a single column, there is no way for a non-key attribute to depend on only part of the key — 2NF is satisfied by construction

3NF — Third Normal Form

Are there non-key attributes that depend on another non-key attribute?

- None. Data that could have created one category names/descriptions, location details, staff details was separated into its own reference table (COMPLAINT_CATEGORIES, LOCATIONS, STAFF) rather than repeated inside COMPLAINTS, so COMPLAINTS only holds foreign keys to that data, not copies of it. One borderline case worth noting: complaints.flag_reason is only meaningful when is_flagged = TRUE. This is a conditional relationship between two non-key attributes, not a true functional dependency, so it does not break 3NF, but it is worth documenting.

Potential Redundancy / Anomaly Check

Identify at least two possible redundancy or anomaly risks you considered and how your design addresses them.

1. Risk:

Storing category or location details (e.g., the category name or full address) directly inside COMPLAINTS instead of referencing them, which would repeat the same text across many complaint rows and cause update anomalies if a category description or address changed.

Solution:

COMPLAINT_CATEGORIES and LOCATIONS are separate tables, referenced from COMPLAINTS by category_id and location_id, so each category/location is stored once.

2. Risk:

Recording status history as extra columns in COMPLAINTS (status_1, date_1, status_2, date_2, …), which would be a repeating group, waste space for complaints with few status changes, and cap how many changes could ever be recorded.

Solution:

STATUS_LOGS is a separate table with one row per status change, linked to COMPLAINTS by complaint_id, so history can grow without altering the schema.

PART 9 — INTEGRITY CONSTRAINTS

Identify the constraints your PostgreSQL implementation should use to protect data integrity.

Constraint Type	Where It Will Be Used	Why It Is Needed
PRIMARY KEY	All tables: users.user_id, staff.staff_id, complaint_categories.category_id, locations.location_id, complaints.complaint_id, complaint_assignments.assignment_id, status_logs.status_log_id, attachments.attachment_id	Uniquely identifies each record and prevents duplicate primary identifiers.
FOREIGN KEY	complaints.user_id → users.user_id	Ensures that every complaint belongs to an existing resident account.
FOREIGN KEY	complaints.category_id → complaint_categories.category_id	Ensures that every complaint uses a valid complaint category.
FOREIGN KEY	complaints.location_id → locations.location_id	Ensures that every complaint is associated with an existing location.
FOREIGN KEY	complaint_assignments.complaint_id → complaints.complaint_id	Ensures that an assignment refers to an existing complaint.

FOREIGN KEY	complaint_assignments.staff_id → staff.staff_id	Ensures that only existing staff members can be assigned to complaints.
FOREIGN KEY	status_logs.complaint_id → complaints.complaint_id	Ensures that every status history record belongs to an existing complaint.
FOREIGN KEY	status_logs.staff_id → staff.staff_id	Ensures that status updates are associated with an existing staff member.
FOREIGN KEY	attachments.complaint_id → complaints.complaint_id	Ensures that every attachment belongs to an existing complaint.
NOT NULL	Required fields such as user names, emails, passwords, complaint category, location, description, status, assignment IDs, and attachment metadata	Prevents required information from being missing.
UNIQUE	users.email and staff.email	Prevents multiple accounts from using the same email address.
UNIQUE	complaint_categories.category_name	Prevents duplicate complaint category names.
UNIQUE	complaint_assignments(complaint_id, staff_id)	Prevents the same staff member from being assigned to the same complaint more than once.
CHECK	complaints.status	Restricts complaint status to Pending, In Progress, or Resolved.
CHECK	status_logs.status	Ensures status history contains only valid complaint statuses.
CHECK	users.account_status and staff.account_status	Restricts account status to valid values such as Active and Inactive.
CHECK	complaints.is_flagged	Ensures the field contains only Boolean values (TRUE/FALSE).
CHECK	attachments.file_size_bytes	Prevents negative or invalid file sizes from being stored.
CHECK	attachments.file_type	Can restrict attachments to accepted image MIME types such as JPEG and PNG.
CHECK	complaints.resolved_at	Ensures a resolved complaint has a resolution timestamp.
DEFAULT	complaints.status	Automatically assigns Pending to a newly submitted complaint.
DEFAULT	complaints.is_flagged	Automatically sets new complaints to FALSE.
DEFAULT	Account statuses in users and staff	Automatically sets newly created accounts to Active.
DEFAULT	created_at, submitted_at, assigned_at, updated_at, and uploaded_at	Automatically records the date and time of important database events.

PART 10 — SYSTEM ARCHITECTURE

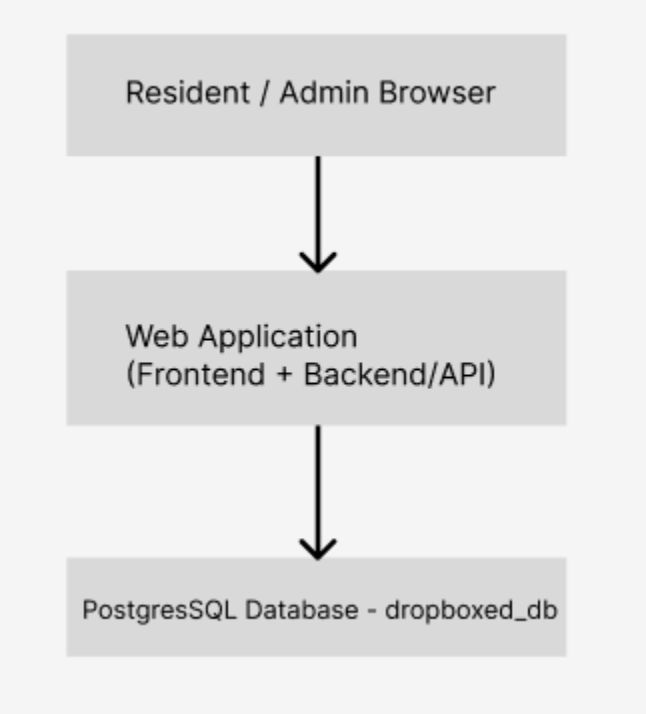
Design the overall architecture of the proposed application. Identify how users, the application, and PostgreSQL interact.

A common reference model is:

Users → Presentation / Client → Application / API → PostgreSQL Database

Your actual architecture may differ if you have a valid reason. The group must justify the architecture.

Architecture Diagram



Flow: Residents and admins interact through the same web application (with role-based views); the application server handles business logic (e.g., validating complaints, enforcing status transitions) and sends SQL queries to PostgreSQL, which stores and returns the data.

Architecture Components

Component / Layer	Technology / Tool	Purpose
Client/Presentation	Web browser (HTML/CSS/JS front-end, e.g., accessed via desktop or mobile)	Allows residents to submit/view complaints and admins to manage them through forms and dashboards
Application/API Layer	Backend framework (e.g., Java Spring Boot, or Node.js/Express)	Handles authentication, business logic (status transitions, flagging duplicates), and communicates

		between the client and the database
Data Access Layer	JDBC / ORM (e.g., Hibernate or a query builder)	Manages connections and queries to PostgreSQL, translating application objects into SQL
Database	PostgreSQL 18.6 (`dropboxed_db`)	Stores all persistent data — users, staff, complaints, categories, locations, assignments, status logs, attachments
Authentication	Password hashing (e.g., bcrypt) at the application layer	Secures resident and staff login credentials
File Storage	Local/server file storage referenced by `attachments.file_path`	Stores uploaded photo evidence of water issues, with metadata tracked in the database

Architecture Justification

Why is this architecture appropriate for your application?

- This three-tier architecture (Client → Application/API → PostgreSQL) is appropriate for DropBoxed because the application has two distinct user roles (residents and staff/admin) that need different views and permissions, which are best enforced in an application layer rather than directly at the database. Separating the application logic from the database also allows business rules — such as restricting complaint status to Pending/In Progress/Resolved, or flagging potential duplicate complaints — to be validated before data reaches PostgreSQL, while PostgreSQL itself enforces integrity through primary keys, foreign keys, and constraints. This separation keeps the system maintainable and scalable: the group can modify the front-end or add features (e.g., notifications) without changing the underlying database structure, and the database can be reused for the subsequent database administration activities as required by the project.

PART 11 — POSTGRESQL DATABASE PLAN

Prepare the database structure that will be used by the group for succeeding database administration activities.

Database Name	dropboxed_db
Schema Name	dropbox
PostgreSQL Version	18.6
Database Owner	postgres
Number of Planned Tables	8
Expected Initial Data Volume	low (<1000 rows)
Expected Growth / Future Data	moderate (50,000 to 500,000 rows)
Current Implementation Status	design phase

Database Naming Convention

Use consistent, descriptive names. Suggested database format:

`grpXX_projectdb`

Example:

`grp03_librarydb`

Avoid generic names such as test, database1, finaldb, or other names that do not identify the project.

Initial SQL / DDL Plan

List the major DDL operations that will be required. You may submit the actual SQL separately or attach it later.

- CREATE DATABASE / database creation step
- CREATE SCHEMA / schema creation step, if applicable
- CREATE TABLE statements
- PRIMARY KEY definitions
- FOREIGN KEY definitions
- NOT NULL / UNIQUE / CHECK / DEFAULT constraints
- Other required database objects

PART 12 — DATABASE HANDOFF SHEET

This sheet is intended for the database administrator who will work with the group's database.

Project / Application	DropBoxed: An Environmental Complaint Application for Household Water Quality Monitoring in San Pedro
Group	riparion
Database Name	dropboxed_db
Schema Name	dropboxed
PostgreSQL Version	18.6
Database Owner	postgres
Application Type	Web-based complaint reporting and tracking system
Number of Tables	8 (users, staff, complaint_categories, locations, complaints, complaint_assignments, status_logs, attachments)
Number of Relationships	7 (users–complaints, complaint_categories–complaints, locations–complaints, complaints–attachments, complaints–status_logs, staff–status_logs, complaints↔staff via complaint_assignments)
Important Database Objects	Primary/foreign keys on all tables; UNIQUE constraints on users.email, staff.email, complaint_categories.category_name, and complaint_assignments(complaint_id, staff_id);

	CHECK constraints on status fields, account_status, is_flagged, and file metadata; DEFAULT values for status, is_flagged, and timestamp columns
Current Database Status	Design complete; schema planned and reviewed for normalization (1NF–3NF); DDL implementation pending
Known Issues / Limitations	Duplicate/spam complaint detection is handled at the application level, not enforced by the database; file storage is local, not cloud-based, which may limit scalability
Future Administration Concerns	Growth of status_logs and attachments tables over time will need indexing/archiving strategy; backup and recovery procedures still need to be defined for succeeding activities

PART 13 — AI USAGE STATEMENT

AI assistance is permitted, but the group must transparently document how it was used.

AI Tool	Purpose / Task	What the Group Changed / Verified	Member(s) Responsible
Claude	Drafted and organized Part 3 (Requirements Analysis) and Part 8 (Normalization Review) content	Group reviewed and verified the analysis matched the schema and design decisions	Dela Cruz, Jan Emmanuel A.
Gemini	Drafted Part 5 (Conceptual Design / ERD)	The group reviewed and verified the ERD for accuracy, consistency, and alignment with the database requirements and schema. The final ERD was manually created and refined using draw.io	Santiago, Jhayciel D.
Claude	Task Summarization	This was used by the group as a guide, blueprint, and direction for the consistency and organization of our delegated task	Abalo, Joshua
ChatGPT	Drafting Parts 4, 7, and 9	Assisted in organizing the Business Rules, Data Dictionary, and Integrity Constraints based on the DropBoxed database design.	Reyes, Denver R.
Gemini	Drafting Parts 2, 10, and 12	Assisted in organizing the Application Proposal,	Valera, Rhaine Cyrel M.

		Database Architecture, and Backup & Recovery Plan based on the DropBoxed system requirements.	

Sample statement:

“ChatGPT was used to generate an initial CRUD implementation and assist with debugging SQL. The group reviewed, modified, tested, and integrated the generated code. The database requirements, business rules, ERD, schema, constraints, and architecture were planned and justified by the group.”

PART 14 — FINAL SUBMISSION CHECKLIST

Task / Requirement	Status	Evidence / Notes
Group information completed	DONE	Group members, project title, application description, and assigned tasks were completed.
PostgreSQL installation/environment checklist completed	DONE	Documented Windows 11, PostgreSQL 16, and local environment details in Part 1.
PostgreSQL verification screenshots included	DONE	Attached the 5 required screenshots (psql version, running service, pgAdmin connections).
Application proposal completed	DONE	Defined DropBoxed problem statement, objectives, and scope.
At least 8 functional requirements documented	DONE	Listed 8 prioritized features including ticket submission, assignment, and status tracking.
Requirements sources documented	DONE	Noted informal interviews, SLA review, and SME discussions.
Data requirements identified	DONE	Detailed major data categories (Subscribers, Tickets, etc.).
At least 8 business rules documented	DONE	Outlined 8 rules covering cardinality, status defaults, and ticket assignments.
ERD completed and readable	DONE	Inserted diagram showing the 5 core entities and their 1:N relationships.
Relational schema completed	DONE	Mapped the conceptual design into 5 schemas with PKs and FKs defined.
Data dictionary completed	DONE	Documented column properties, data types, and nullability for the TICKET table.

Normalization review completed	DONE	Verified 1NF, 2NF, and 3NF compliance and explained the category table separation.
Integrity constraints documented	DONE	Specified PK, FK, NOT NULL, UNIQUE, CHECK, and DEFAULT rules.
System architecture diagram completed	DONE	Defined a 3-tier architecture (Presentation, API, Database).
Architecture justified	DONE	Explained the security and separation of concerns benefits of the 3-tier model.
PostgreSQL database plan completed	DONE	Documented details for dropboxed_db, expecting moderate growth.
Database handoff sheet completed	DONE	Summarized the 8-table system and known limitations for the DBA.
AI usage statement completed	DONE	Disclosed AI assistance for schema structuring and DDL drafting.
Document reviewed for consistency	DONE	Confirmed ERD, schemas, and business rules align perfectly with the proposed system.

Assessment Criteria

Criteria	Weight	What Is Expected
PostgreSQL setup and verification	10%	Environment is installed/verified and evidence is complete.
Requirements analysis	10%	Requirements are specific, realistic, and traceable to the proposed system.
Business rules	10%	Rules are meaningful and influence the design.
ERD / conceptual design	15%	Entities, attributes, relationships, cardinality, and M:N resolution are appropriate.
Relational schema / normalization	15%	Schema is consistent, normalized appropriately, and logically justified.
Data dictionary / constraints	10%	Columns and integrity rules are clearly documented.
System architecture	10%	Architecture is clear and appropriately justified.
PostgreSQL database planning	10%	Database structure and implementation plan are realistic and manageable.
Documentation quality	5%	Submission is organized, readable, complete, and consistent.
AI transparency / responsible use	5%	AI use is disclosed and generated work is reviewed and understood.

Important Reminders

- The application idea may be original; feasibility and database requirements matter more than complexity.
- Do not start by asking AI to “build the whole system.” Start with requirements and design.

- Your ERD, relational schema, data dictionary, and PostgreSQL implementation must agree with one another.
- Do not expose passwords, connection secrets, or other sensitive credentials in the submission.
- The assigned PostgreSQL database will be used for succeeding database administration activities, so keep your database organized and documented.
- A database is not an “install once and forget forever” system. Future activities will use the project database to practice administration tasks.

Project Flow

PostgreSQL Setup → Requirements → Business Rules → ERD → Relational Schema → Normalization → Constraints → Architecture → Database Plan → Implementation → Database Administration

Design carefully. Document everything. Then build.